SCA-MQDSA: Side-Channel Analysis of Multivariate Digital Signature Implementations

N.K. Vishwaajith*
Indian Institute of Technology Bombay
Mumbai, Maharastra, India
23b1038@cse.iitb.ac.in

Anindya Ganguly*
Indian Institute of Technology Kanpur
Kanpur, Uttar Pradesh, India
anindyag@cse.iitk.ac.in

Debranjan Pal
LNM Institute of Information Technology
Jaipur, India
debranjan.crl@gmail.com

Trevor Yap
Nanyang Technological University
Singapore, Singapore
trevor.yap@ntu.edu.sg

Puja Mondal
Indian Institute of Technology Kanpur
Kanpur, Uttar Pradesh, India
pujamondal@cse.iitk.ac.in

Suparna Kundu
COSIC, KU Leuven
Leuven, Belgium
suparna.kundu@esat.kuleuven.be

Sayandeep Saha
Indian Institute of Technology Bombay
Mumbai, India
sayandeepsaha@cse.iitb.ac.in

Shivam Bhasin
Temasek Laboratories, Nanyang Technological University
Singapore
sbhasin@ntu.edu.sg

Ingrid Verbauwhede
COSIC, KU Leuven
Leuven, Belgium
ingrid.verbauwhede@esat.kuleuven.be

Angshuman Karmakar
Indian Institute of Technology Kanpur
Kanpur, Uttar Pradesh, India
angshuman@cse.iitk.ac.in

Abstract

The rapid progress of Internet-of-Things (IoT) systems and network protocols has strengthen the demand for digital signature schemes with compact signatures and low computational overhead. However, standardized post-quantum signature schemes, such as ML-DSA, SLH-DSA, and Falcon, incur relatively large signature sizes, which limit their practicality on resource-constrained devices (RCD). To address this challenge, NIST recalled the post-quantum digital signature standardization process. It reopened the interest in alternative constructions. Signature schemes based on multivariate quadratic equations have emerged as promising candidates due to their comparatively small signature sizes and efficient verification. At the same time, these schemes are often deployed on platforms with limited physical protections, making side-channel security a critical concern.

Four multivariate-based signature schemes, such as UOV, MAYO, QR-UOV, and SNOVA, were reached in the second round of the NIST post-quantum signature standardization process for further evaluation. In this work, we analyze implementations of multivariate digital signature algorithms submitted to the NIST process, with a particular focus on the MAYO signature scheme. We present an *inexpensive* side-channel attack targeting nibble-sliced implementations of MAYO. Our attack operates under a minimal threat model: it does not require physical possession of the target device and succeeds using leakage obtained from a single signing execution. We perform our attacks on a ChipWhisperer-Lite platform with a 32-bit STM32F3 ARM Cortex-M4 target mounted on a CW308 UFO board, and we add the corresponding reference code along with target traces.

Keywords

Side-channel analysis, Post-quantum Cryptography, Multivariate cryptography, Single-trace, Non-profiling, Mayo

1 Introduction

MQ-DSAs and their importance. Multivariate quadratic-based digital signature algorithms (MQ-DSAs) are believed to be secure against quantum adversaries. They generally follow the hash-and-sign paradigm, which relies on structural designs built upon a trapdoor multivariate quadratic (MQ) system. The hardness of solving systems of MQ equations over finite fields is known to be NP-complete [16]. MQ-DSAs proposed in the literature, however, provide significantly shorter signatures and enable faster signing and verification compared to the NIST-standardized lattice- and hash-based schemes [1], namely Dilithium [11, 22], Falcon [12], and SPHINCS+ [23]. For instance, the classical UOV scheme achieves a signature size of around 96 bytes for 128-bit security, which is significantly smaller than most standardized alternatives. These features make them attractive for constrained or embedded environments. Prominent examples include UOV [8, 17], MAYO [7], SNOVA [27], and QR-UOV [13]. These designs are second-round candidates in the NIST additional signature standardization process [24].

Physical attacks. Cryptographic protocols deployed in real-world devices are often vulnerable to physical attacks that exploit information leaked through side channels such as timing [20], power consumption [19], electromagnetic radiation, [14] or fault behaviour.

*Both authors contributed equally to this research.

These attacks, broadly referred to as side-channel analysis (SCA), aim to extract secret information by analysing such unintended leakages. A common form is correlation power analysis (CPA), where the attacker correlates power traces with hypotheses about intermediate values to recover secret keys. In more advanced settings, single-trace attacks may succeed using machine learning or algebraic techniques that exploit structural dependencies within a single execution. SCAs are typically categorized into profiling and non-profiling attacks. In profiling attacks, the adversary builds a leakage model using a similar device with known keys (e.g., template attacks). In contrast, non-profiling attacks assume no prior leakage characterization and directly infer the key from the observed traces. These physical attacks bypass traditional black-box assumptions of cryptographic security and pose serious threats, especially to embedded systems.

Physical attacks on MQ-DSAs. Various physical attacks were proposed against MQ-DSAs. These state-of-the-art attacks are summarized in the SoK [2]. We are only focused on side-channel (SC) attacks. We begin with a correlation power attack (CPA) on MQ-DSAs, like Rainbow, UOV, Mayo. Rainbow was a third-round candidate in the NIST post-quantum signature standardization process [10]. UOV is the classical construction in the MQ family [8, 9, 17], while MAYO is one of the most recent designs [7].

CPA on Rainbow. Optimized Rainbow implementation uses equivalent keys with fixed central maps and structured affine transformations to reduce key size and speed up computation. However, this optimization introduces SC leakages during signature generation. An attacker using correlation power analysis (CPA) can exploit matrix–vector multiplications in the affine transformations. Once one affine transformation is known, algebraic methods can be used to recover full key. Park *et al.* [25] demonstrated the recovery of the full secret key from only 30 traces on an Atmel XMEGA128 processor and proposed masking and hiding techniques as potential countermeasures. Concurrently, Pokorný *et al.* [26] validated a similar CPA attack on an STM32F303 processor, achieving subkey recovery with probabilities ranging from 0.75 to 0.95 using 40 to 475 traces, and recommended a multiplicative masking strategy to mitigate the threat.

CPA on UOV and Mayo. Park et al. [25] extended the CPA attack on Rainbow to UOV (for equivalent keys). As mentioned, this attack is a multi-trace attack, and they have not experimentally validated their claims. Aulbach *et al.* first proposed a single-trace SCA on UOV. This is a profiling attack. It targets the signing algorithm to recover all vinegar variables, after which algebraic techniques such as the Kipnis–Shamir attack [18] are used to recover the secret key. A similar approach can be used against MAYO. Jendral *et al.* [15] performed this attack on an earlier implementation of MAYO, employing deep learning–assisted power analysis.

Research gaps. Non-profiling SCAs are considered more practical. This model does not require access to a clone of the target device. So far, the attacks proposed against MQ-DSAs have been single-trace SCAs in the profiling setting. However, no such attacks have been demonstrated on MQ-DSAs in the non-profiling setting. In addition, most SCAs against MQ schemes target the multiplication subroutine used during the key generation or signing process. The multiplication subroutine produces high-value intermediates and thus yields strong leakage. Some recent implementations submitted to the NIST process have modified the multiplication algorithms [7]. As a result, the leakage profiles in these newer implementations can differ substantially from earlier scenarios. It is therefore crucial to empirically study how leakages arise in the newest implementations. Thus, we ask the following question in this context.

Question: Can non-profiling SCAs be performed against these new MQ-DSA implementations without relying on deep-learning techniques?

1.1 Our Contribution

As stated earlier, the primary objective of this work is to study SCA on implementations of MQ-DSAs. We focus on MAYO as a representative case study to illustrate the attack methodology. However, the proposed approach is not specific to MAYO and can be extended to other MQ-DSAs. Our results answer the research question posed earlier in the affirmative.

SCA on MQ-DSAs: *answer to the question.* Our contributions are organized into three main components.

(1) We revisit state-of-the-art SCA on MQ-DSAs in Section 2 and identify the corresponding leakage points. To mount our attack, we target the `nibble-sliced` implementation of MAYO SL-1. This implementation represents the most recent version and was submitted [5] to the second round of the NIST-PQC standardization process [24].
(2) The setup of our attack is deliberately simple and does not require access to a clone of the target device, which reflects the practical relevance. Notably, our attack recovers all coefficients of the oil matrix using only side-channel measurements. We do not rely on machine learning techniques or algebraic key-recovery algorithms. To the best of our knowledge, this is the first SCA that achieves full secret-key recovery for the target function `P1P1t_times_O` using purely physical attack.
   We observe that, during a single execution of MAYO SL-1, the underlying $\mathbb{F}_{16}$ multiplication subroutine is invoked 240,240 times, yielding a sufficient number of sub-traces to enable a non-profiled single-trace SCA. Using this approach, we recover individual coefficients of the oil matrix. To recover the entire matrix, we exploit the sequential structure of the computation: after recovering an entry $O_{r,c}$, we update the accumulator accordingly and proceed to recover $O_{r,c+1}$. The complete key-recovery methodology is described in Section 3. All experiments are conducted on a ChipWhisperer-Lite platform using a 32-bit STM32F3 ARM Cortex-M4 target mounted on a CW308 UFO board. We provide the corresponding reference implementation and collected power traces.
(3) We extend our analysis to other MQ-DSA implementations, including UOV. Depending on the parameter set, UOV uses arithmetic over $\mathbb{F}_{16}$ or $\mathbb{F}_{256}$. We analyze leakage from both field multiplication routines and demonstrate full recovery of the oil matrix using the same sequential key-recovery strategy. Our attack framework is also applicable to related schemes such as QR-UOV and SNOVA.

2 Preliminaries

Notations. We use bold capital letters to denote matrices, such as $\mathbf{P}$ and $\mathbf{O}$, and bold lowercase letters to denote column vectors, such as $\mathbf{x}$, $\mathbf{y}$, and $\mathbf{o}$. We use lowercase letters such as q, n, m, v, o to denote natural numbers. Here, q is the size of the finite field, n is the number of unknowns, m is the number of quadratic constraints, v is the number of vinegar variables, and o is the dimension of the secret oil subspace O. The notation $\mathbf{0}_m$ denotes m zero entries in $\mathbb{F}_q^m$, and $\mathbf{I}_m$ is a $m \times m$ dimension invertible matrix. The time complexity of multiplying two square matrices of size n is denoted by $O(n^\omega)$, where $2 \leq \omega < 3$. The notation $\mathrm{Upper}(\mathbf{A})$ defines a upper triangular matrix, which has the non-zero diagonal entries.

Quadratic maps, such as $\mathcal{P}$, $\mathcal{DP}$, are written using a calligraphic font. For a quadratic map $\mathcal{P} : \mathbb{F}_q^n \to \mathbb{F}_q^m$, we define its *differential polar form* (denoted by $\mathcal{DP}$) as the symmetric bilinear map: $\mathcal{DP}(\mathbf{x}, \mathbf{y}) = \mathcal{P}(\mathbf{x} + \mathbf{y}) + \mathcal{P}(\mathbf{x}) + \mathcal{P}(\mathbf{y})$. The form $\mathcal{DP}_\mathbf{x}$ defines the linear system $\mathbf{y}$.

We use the term *MQ-DSA* to refer to Hash-and-Sign signatures based on multivariate cryptography that use Oil-Vinegar (OV) as the underlying trapdoor. Our notation covers all second-round candidates from the NIST additional standardization process.

2.1 Side-Channel Analysis (SCA)

Side-channel analysis (SCA) exploits physical leakages during the execution of cryptographic algorithms in a device. These leakages can be observed during the instantaneous power consumption, electromagnetic (EM) radiation, timing behaviour, etc. These leakages of sensitive intermediate variables can be modelled through a leakage function, which is correlated with the actual secret value via statistical dependencies such as the Hamming weight (HW) or Hamming distance. In practice, SCAs bypass the underlying mathematical hardness assumption and directly target the implementation layer. Their relevance is therefore immediate for real-world devices (such as smartcards, secure microcontrollers, and PQC accelerators), where even PQ-secure primitives can be compromised if the leakage model is sufficiently accurate and the implementation remains unprotected.

Profiling vs. Non-Profiling Attack Paradigms. Side-channel (SC) adversaries are generally characterized by the control of the target device and its leakages. In *profiling attacks*, the adversary possesses an identical device on which arbitrary computations can be performed to build an accurate leakage model. On the other hand, *non-profiling attacks* assume no such device and rely on generic leakage hypotheses and statistical correlations derived directly from the target implementation. Both settings are realistic: profiling applies to mass-produced or publicly available hardware, while non-profiling better models single-instance or high-assurance deployments where duplication is infeasible.

Correlation Power Analysis (CPA). CPA relies on the observation that the power consumption of hardware depends on the data being processed. In practice, this dependence is often modelled using the HW or Hamming distance leakage model, which assumes that power consumption is proportional to the number of bits set to '1' in internal registers. CPA evaluates different key hypotheses by predicting the corresponding power consumption of selected intermediate values and comparing these predictions with measured power traces using Pearson's correlation coefficient.

Intermediate Value Selection. Select an intermediate value computed as a target function $TF(d_i, k_j)$ of known, non-constant data d_i and the secret key or secret key-related information k_j.

Power Measurement. Measure the power consumption of the target function during execution and identify the time samples corresponding to the computation of the selected intermediate value.

Hypothetical Intermediate Computation. Compute hypothetical values of the selected intermediate for all possible candidates of the secret sub-key k_j using the known data.

Leakage Modeling. Map the hypothetical intermediate values to predicted power consumption using the HW or Hamming distance leakage model.

Correlation-Based Key Distinguishing. Compute the Pearson correlation coefficient between predicted and measured power values and identify the key hypothesis that maximises the correlation.

2.2 Multivariate Quadratic-based Digital Signature Algorithms (MQ-DSAs)

We focus on DSAs in multivariate cryptography that follow the hash-and-sign paradigm. The security of these constructions is based primarily on the hardness of the MQ problem. Since these designs rely on the inversion of a trapdoor function, an additional and comparatively weaker assumption is typically involved in their security analysis.

2.2.1 *Hard problems.*

We define the MQ problem and the extended isomorphism problem in this context.

Definition 2.1 (Multivariate Quadratic (MQ) Problem). *Suppose $\mathcal{P} : \mathbb{F}_q^n \to \mathbb{F}_q^m$ is a MQ map defined over $\mathbb{F}_q$. It has n variables and m quadratic equations. The MQ problem asks to find a vector $\mathbf{s} \in \mathbb{F}_q^n$ such that $\mathcal{P}(\mathbf{s}) = \mathbf{t}$ for a given non-zero target vector $\mathbf{t} \in \mathbb{F}_q^m$, and MQ map $\mathcal{P}$. It is known to be NP-complete [16].*

Definition 2.2 (Extended Isomorphism Problem (EIP)). *Let $\mathcal{P} : \mathbb{F}_q^n \to \mathbb{F}_q^m$ is a public MQ map, and let O denote a o-dimensional secret subspace on which $\mathcal{P}$ vanishes, i.e., $\mathcal{P}(O) = \mathbf{0}_m$. The* EIP *asks to find an equivalent o-dimensional subspace $\tilde{O}$ such that $\mathcal{P}(\tilde{O}) = \mathbf{0}_m$. Till now, no hardness classification is known for this problem.*

2.2.2 *Constructions for MQ-DSAs.*

The Oil-vinegar technique is the core of all MQ-DSAs following hash-and-sign paradigm [17]. It first samples a secret subspace O of dimension o. This can be done by sampling a seed seed_{sk} and expanding to an $(n-o) \times o$ dimension matrix $\mathbf{O}$, whose column vectors are linearly independent. Then it constructs a public multivariate quadratic map or *UOV map* $\mathcal{P} : \mathbb{F}_q^n \to \mathbb{F}_q^m$, so that $\mathcal{P}(\mathbf{O}) = \mathbf{0}$. It has m-homogeneous quadratic polynomials and n variables. Each polynomial in $\mathcal{P}$ is represented as a matrix $(\mathbf{P}_1, \mathbf{P}_2, \cdots, \mathbf{P}_m)$, where each $\mathbf{P}_i \in \mathbb{F}_q^{n \times n}$. Due to the structure of $\mathcal{P}$, each $\mathbf{P}_i$ can be further split into three sub-matrices. Two of these matrices $\mathbf{P}_i^{(1)} \in \mathbb{F}_q^{(n-m)\times(n-m)}$ (upper triangular) and $\mathbf{P}_i^{(2)} \in \mathbb{F}_q^{(n-m)\times m}$ can be generated deterministically from a random seed seed_{pk}. The other matrix $\mathbf{P}_i^{(3)}$ can be set

as Upper $\left(-\mathbf{O}^\top \mathbf{P}_i^{(1)} \mathbf{O} - \mathbf{O}^\top \mathbf{P}_i^{(2)}\right)$ by exploiting the fact $\mathcal{P}(\mathbf{O}) = \mathbf{0}_m$.

$$\bar{\mathbf{O}}^\top \mathbf{P}_i \bar{\mathbf{O}} = \begin{pmatrix} \mathbf{O}^\top & \mathbf{I}_o \end{pmatrix} \begin{pmatrix} \mathbf{P}_i^{(1)} & \mathbf{P}_i^{(2)} \\ \mathbf{0} & \mathbf{P}_i^{(3)} \end{pmatrix} \begin{pmatrix} \mathbf{O} \\ \mathbf{I}_o \end{pmatrix} = \mathbf{0}_m$$

To make signature generation faster, the scheme pre-computes a matrix $\mathbf{L}_i$ for each i as

$$\mathbf{L}_i = (\mathbf{P}_i^{(1)} + \mathbf{P}_i^{(1)\mathsf{T}}) \cdot \mathbf{O} + \mathbf{P}_i^{(2)}. \tag{1}$$

In the end, it publishes compact secret key $csk = \text{seed}_{\text{sk}}$, and public key as $cpk = (\text{seed}_{\text{pk}}, \{\mathbf{P}_i^{(3)}\}_{i=1}^m)$. Then these key can expanded as $esk = \left(\text{seed}_{\text{sk}}, \mathbf{O}, \{\mathbf{P}_i^{(1)}, \mathbf{L}_i\}_{i=1}^m\right)$, and $epk = \{\mathbf{P}_i\}_{i=1}^m)$. The signature generation procedures in UOV and MAYO differ in their design; however, the verification algorithms are the evaluation of a public map.

Unbalanced Oil-Vinegar (UOV)[8, 17]. The UOV scheme was introduced by Kipnis *et al.* [17]. This work follows the design and specification provided by Beullens *et al.* [8]. In UOV, the dimension of the secret subspace $\mathbf{O}$ equals the number of equations, i.e., $o = m$. According to the specification [8], the UOV scheme consists of three key algorithms UOV.CompactKeyGen, UOV.Sign, and UOV.Verify. These main algorithms use two sub-algorithms, UOV.ExpandSK : csk $\rightarrow$ esk and UOV.ExpandPK : cpk $\rightarrow$ epk. These sub-algorithms are responsible for expanding the compressed secret and public keys. For implementation flexibility, UOV supports three variants. The `classic` version uses the expanded keys (esk, epk) directly, providing higher efficiency in signature generation and verification at the cost of larger key sizes. The pkc (public-key-compressed) version executes UOV.ExpandPK in UOV.Verify, resulting in smaller public keys but slower verification. Finally, the pkc+skc (doubly-compressed) version performs both UOV.ExpandSK and UOV.ExpandPK in UOV.Sign and UOV.Verify, respectively, which reduces key sizes and speeds up key generation but makes signing slower.

MAYO Signature. [7] The main difference between the UOV and MAYO constructions lies in the dimension of their secret subspaces. In contrast to UOV, where the oil-space dimension equals the number of quadratic equations, MAYO uses a significantly smaller oil-space dimension. For instance, in the MAYO-SL1 parameter set, the oil dimension is $o = 8$ while the number of equations is $m = 78$. To bridge this gap, MAYO introduces a *whipping map* ensuring $m \approx k \cdot o$, where k denotes the whipping parameter.

MAYOwhipping map. Suppose $\mathcal{P}$ be an (n, m)-UOV map that vanishes on an o-dimensional secret subspace. The MAYO construction extends this using a *whipping map*, denoted by $\mathcal{P}^* : \mathbb{F}_q^{nk} \rightarrow \mathbb{F}_q^m$, defined as

$$\mathcal{P}^*(\mathbf{s}_1, \ldots, \mathbf{s}_k) = \sum_{i=1}^{k} \mathbf{E}_{i,i}\, \mathcal{P}(\mathbf{s}_i) + \sum_{i=1}^{k} \sum_{j=i+1}^{k} \mathbf{E}_{i,j}\, \mathcal{DP}(\mathbf{s}_i, \mathbf{s}_j).$$

Here, $\mathbf{E}_{i,j} \in \mathbb{F}_q^{m \times m}$ are public parameters. During signature generation, the signer first samples $(\mathbf{v}_1, \ldots, \mathbf{v}_k) \in_U \mathbb{F}_q^{nk}$ uniformly at random, and then computes $(\mathbf{o}_1, \ldots, \mathbf{o}_k) \in \mathcal{O}^k$ such that $\mathcal{P}^*(\mathbf{v}_1 + \mathbf{o}_1, \ldots, \mathbf{v}_k + \mathbf{o}_k) = \mathbf{t}$. Then it outputs $(\mathbf{s}_1, \ldots, \mathbf{s}_k) = (\mathbf{v}_1 + \mathbf{o}_1, \ldots, \mathbf{v}_k + \mathbf{o}_k)$ as the signature.

2.3 SCA in MQ-DSAs

In MQ-DSAs, two equivalent secret-key representations are commonly used. The classical approach, dominant before 2020, defines the public map as a composition of affine and central maps, typically $\mathcal{P} = \mathcal{S} \circ \mathcal{F} \circ \mathcal{T}$, which conceals the structure of the central polynomial. More recent schemes follow the subspace approach [7, 8], introduced by Beullens[4], where the secret key is a subspace on which the public map vanishes. This formulation leads to significantly smaller public keys while remaining mathematically equivalent to the compositional view.

As discussed in Section 2.1, the SCA focus on functions that process secret-dependent data together with public inputs and thereby leak information about the secret. In this analysis, we therefore indeitfy the functions executed during the MQ-DSA signature algorithm that involve such secret-dependent computations.

Side-Channel Analysis on Compositional MQ-DSA Implementations. MQ-DSAs such as Rainbow and UOV rely on the trapdoor construction $\mathcal{P} = \mathcal{T} \circ \mathcal{F} \circ \mathcal{S}$, where $\mathcal{F}$ denotes a structured quadratic map and $\mathcal{S}, \mathcal{T}$ are secret invertible affine transformations. The signing algorithm explicitly evaluates the inverse maps $\mathcal{T}^{-1}$, $\mathcal{F}^{-1}$, and $\mathcal{S}^{-1}$. These computations involve secret-dependent finite-field arithmetic, which gives rise to exploitable physical leakage under Correlation Power Analysis (CPA).

Park et al. [25] focused on the practical vulnerability of the ARM Cortex-M4 implementation of Rainbow and UOV schemes. The main idea is that the matrix-vector multiplication $\mathbf{y} = \mathbf{T} \cdot \mathbf{x}$, which is required to map variables from the central map back to the public-key space—is highly susceptible to CPA. Because $\mathbf{T}$ is the matrix corresponding to the secret affine transformation $\mathcal{T}$ over a finite field (typically $\mathbb{F}_{16}$ or $\mathbb{F}_{256}$). Here, the power consumption of each multiplication leaks the Hamming weight of intermediate values. By targeting the first few rounds of the signing process, an attacker can recover the secret rows of $\mathbf{T}$ one-by-one.

Pokorný *et al.* [26] performed a new attack and they exploited the "*layered*" structure of Rainbow. Here, the attacker can focus on the first layer to recover a portion of the secret key and then use algebraic techniques to remove the layers, significantly reducing the complexity of a full-key recovery compared to a purely mathematical cryptanalysis.

Aulbach et al. [3] introduced a single-trace profiling SC attack against UOV-based signature schemes. Their attack targets the optimized UOV implementation proposed in [9] and demonstrates that a single SC trace of the signing procedure suffices to recover vinegar variables.

SCA on Subspace-based MQ-DSA Implementations. Jendral *et al.* extended the attack methodology of Aulbach *et al.* [3] to the MAYO signature scheme [15]. Additionally, the survey by Aulbach *et al.* [2] systematically identifies potential SC attack surfaces in MQ-DSAs.

Recent SCA and Sensitivity Analyses of MQ-DSAs. In the recent second round of NIST additional call, four candidates from the MQ-DSA family were reached [24], namely UOV [8], Mayo [7], QR-UOV [13], and SNOVA [27]. To assess the SC security of these schemes, it is necessary to analyse their concrete implementations. Although these constructions share similar underlying mathematical principles, their implementation strategies differ substantially. This leads to distinct SC leakage characteristics. The following

Table 1: Secret-Dependent Operations in the Mayo Signature Algorithm

Expression	Secret-dependent operations
$\mathbf{L}_i \leftarrow (\mathbf{P}_i^{(1)} + \mathbf{P}_i^{(1)\top})\,\mathbf{O} + \mathbf{P}_i^{(2)}$	Dependence on secret oil matrix $\mathbf{O}$ interacting with public matrices.
$\mathbf{v}^\top \mathbf{P}_i^{(1)} \mathbf{v}$	Dependence on secret vinegar vectors $\mathbf{v}_j$ interacting with public matrices.
$\mathbf{s}_i \leftarrow \{\mathbf{v}_i + \mathbf{O}\mathbf{x}_i \,\|\| \, \mathbf{x}_i\}_{1 \le i \le k}$	Multiplication of secret oil matrix $\mathbf{O}$ with known vector $\mathbf{x}$, yielding secret-dependent intermediates.

Table 1 illustrates the possible target points in the signature algorithm. For further details on sensitivity analysis, the reader is referred to [2, 21].

3 Non-profiled Single Trace SCA on MQ-DSAs

In Section§ 2, we described that the target functions of MAYOmentioned in Table 1 are more likely equivalent to other MQ-DSAs. We also observed that the SCA on the implementation of the MAYOsignature scheme [5] is a new research direction. The `bitsliced` version of the MAYOreference implementation [5] (`main` branch of [6]) used `bitsliced_m_vec_mul_add` subroutine. This subroutine runs in the signing algorithm during the computation of each matrix $(\mathbf{P}_i^{(1)} + \mathbf{P}_i^{(1)\top}) \cdot \mathbf{O}$.

Jendral and Dubrova first performed a single-trace SCA [15] on the `bitsliced` implementation of MAYO. Their method is a profiled, deep-learning-assisted analysis based on the HW leakage model. In the profiling phase, they trained neural networks to predict HW of intermediate values from power traces. In the attack phase, they used the trained model to recover a secret oil vector. Then algebraic algorithm is employed to recover an equivalent oil-subspace. Their attack specifically exploits leakage in the `bitsliced_m_vec_mul_add` multiplication and does not apply to `nibble-sliced` version of MAYOimplementation (`nibbling-mayo` branch of [6]). To the best of our knowledge, apart from [15], no SCA have been reported on the `nibble-sliced` MAYOimplementation. Consequently, the side-channel security of the latest `nibble-sliced` MAYOvariant remains an open problem. In this work, we address this gap by analyzing the leakage behavior of its core arithmetic operations, starting with the `nibble-sliced` multiplication subroutine.

3.1 `nibble-sliced` $\mathbb{F}_{16}$ Multiplication.

The `nibble-sliced` MAYOimplementation [5] relies on the subroutine `m_vec_mul_add`, which internally calls `mul_table()`. This function produces a 32-bit encoding of a multiplication constant $b \in \mathbb{F}_2[x]/(x^4 + x + 1)$. Specifically, it expands b across four byte positions using the bit-replication multiplier `0x08040201`, yielding

$$\mathbf{tab} = b \times \texttt{0x08040201} = (8b \,\|\, 4b \,\|\, 2b \,\|\, b).$$

```
static inline uint32_t mul_table(uint8_t b){
    uint32_t x = ((uint32_t) b) * 0x08040201;

    uint32_t high_nibble_mask = 0xf0f0f0f0;

    uint32_t high_half = x & high_nibble_mask;
    return (x ^ (high_half >> 4) ^ (high_half >> 3));
}
```

```
static inline void m_vec_mul_add (int m_vec_limbs, const uint64_t *
     in, unsigned char a, uint64_t *acc) {
    (void) m_vec_limbs;
    uint32_t tab = mul_table(a);

    uint64_t lsb_ask = 0x1111111111111111ULL;

    for(int i=0; i < M_VEC_LIMBS_MAX ;i++){
        acc[i] ^= ( in[i]  & lsb_ask) * (tab & 0xff)
                ^ ((in[i] >> 1) & lsb_ask) * ((tab >> 8)  & 0xf)
                ^ ((in[i] >> 2) & lsb_ask) * ((tab >> 16) & 0xf)
                ^ ((in[i] >> 3) & lsb_ask) * ((tab >> 24) & 0xf);
    }
}
```

Listing 1: C code snippet of `nibble-sliced` $\mathbb{F}_{16}$ Multiplication

In a single invocation, `m_vec_mul_add` algorithm processes 16 elements over the field $\mathbb{F}_{16}$ in parallel by packing them into the input vector **in**. Each of these field elements is multiplied by the same coefficient a in a batched manner, and the resulting products are XOR-accumulated into the corresponding positions of the output vector **acc**. This packed representation enables simultaneous computation of multiple $\mathbb{F}_{16}$ multiplications within a single word-level operation, allowing the routine to efficiently accumulate the results for all 16 field elements in one pass.

Within `P1P1t_times_O`, the function `m_vec_mul_add` processes coefficients from the public matrices $(\mathbf{P}_i^{(1)} + \mathbf{P}_i^{(1)\top})_{i=1}^{m}$ in a batched manner, operating on sixteen $\mathbb{F}_{16}$ elements in parallel per call. Each call multiplies a fixed oil coefficient with the same matrix position across 16 distinct public matrices and accumulates the results into the output vector, and this procedure is repeated up to `m_vec_limbs` times. The `m_vec_limbs` can be calculated as $\lceil m/16 \rceil$. For a fixed oil coefficient $\mathbf{O}(u, k)$, the routine invokes `m_vec_mul_add` exactly `m_vec_limbs` $\cdot\,(v-1)$ times. For instance, under the MAYO-SL1 parameter set with $m = 78$, $v = 78$ and `m_vec_limbs` $= 5$, each oil coefficient triggers 5×77 invocations of `m_vec_mul_add` during the execution of `P1P1t_times_O`.

The description highlights that `m_vec_mul_add` is a key subroutine that runs many times during `nibble-sliced` MAYO execution. To understand how this design leaks information through side channels, we compare it to the `bitsliced` version. Both versions perform the same math, but they use very different internal hardware structures.

3.1.1 Differences between `nibble-sliced` *and* `bitsliced` *multiplications.* These two multiplication routines implement the same $\mathbb{F}_{16}$ vector–scalar product, but their internal mechanisms differ significantly. The routine `m_vec_mul_add` stores each field element in a *nibble-packed* layout inside 64-bit words. It extracts the four coefficient bits of all elements using masked shifts and multiplies each coefficient slice with the corresponding 4-bit value $a \cdot x^k$. Its behavior depends on the *nibble structure* of the inputs and the 4-bit value of a, but all operations occur unconditionally, which reduces explicit bit-dependent switching. The bitsliced routine `bitsliced_m_vec_mul_add` instead distributes the input across four 32-bit slices, where each slice holds the same coefficient bit of many field elements. The scalar a is expanded into four *bit-masks* a_0, a_1, a_2, a_3, and slice k contributes to the output only when $a_k = 1$. This produces *bit-wise conditional execution*, since each mask either activates or suppresses an entire word-level operation.

As a result, the bitsliced routine uncovers strong leakage correlations with the individual bits of a. On the other hand, the nibble-packed routine gives a more uniform control flow and weaker per-bit dependence in SCA. The above distinction in internal data layout and control flow has direct implications for side-channel security. In particular, the different ways in which the scalar a influences intermediate computations determine both the location and the strength of exploitable leakage. While the bitsliced routine exposes explicit bit-level activation patterns that naturally lend themselves to straightforward bit-wise leakage modeling, the nibble-packed routine `m_vec_mul_add` exhibits more subtle, value-dependent leakage due to its uniform execution and packed representation. In the following, we focus on this latter case and demonstrate that, despite its seemingly more regular structure, `m_vec_mul_add` still leaks sufficient information to enable key recovery via correlation power analysis.

3.2 Methodology for CPA to `m_vec_mul_add`.

We present a CPA-based approach to recover a single coefficient of the oil matrix. The setup of the attack is simple and we do not require physical access to the target device, which reflects realistic adversarial settings. Notably, a single execution of the MAYOsigning algorithm is sufficient to reveal secret-dependent information. We then detail the CPA workflow, including the identification of leakage points, a description of our experimental setup, the rationale for why a non-profiling single-trace attack suffices, and the use of standard Pearson correlation to recover the secret key material.

Intermediate value selection. We analyze the SC leakage of the multiplication subroutine `m_vec_mul_add` and identify potential leakage points. The first potential leakage point appears from the secret-dependent computation of `tab = mul_table(a)`. Although `tab` is reused across all loop iterations, it represents the multiplication table of a single $\mathbb{F}_{16}$ element and exhibits a fixed and small Hamming weight (HW). As a result, `tab` alone carries limited exploitable information and does not provide a strong distinguisher for CPA.

We next consider the computations inside the loop, in particular, the individual multiplications of the form

$$\texttt{val}[j] = (\texttt{in}[i] \gg j \;\&\; \texttt{lsb\_mask}) \cdot (\,(\texttt{tab} \gg 8j) \;\&\; 0xf\,). \quad (2)$$

Here $j \in \{0, 1, 2, 3\}$. Let $j = 0$, then the left operand is of the form `0xuuuuuuuu`, where each $\texttt{u} \in \{0, 1\}$ depends only on public data, while the right operand is bounded to a nibble, that is, an element of $\mathbb{F}_{16}$. Consequently, the Hamming weight of the product satisfies

$$\mathrm{HW}(\texttt{val}[0]) = \mathrm{HW}(\text{left operand}) \cdot \mathrm{HW}(\text{right operand}).$$

This structure implies that the leakage from an individual multiplication does not allow a successful CPA distinguisher, since correlations such as $\mathrm{corr}(a, b)$ and $\mathrm{corr}(a, 2b)$ are identical under this leakage model. Therefore, each standalone multiplication inside the loop is not, by itself, a strong leakage point.

The dominant leakage instead originates from the accumulator, which aggregates the results of these secret-dependent multiplications across iterations. This accumulation mixes multiple contributions of `tab` with different public inputs, amplifying the exploitable leakage. Moreover, the accumulator is eventually written back to memory through a power-expensive *store* operation, further increasing its observability.

As a result, the accumulator value emerges as the primary leakage point and a natural target for a CPA attack. This observation is further reinforced by the execution pattern of the implementation by the calling context of `m_vec_mul_add`. As discussed in Section 3.1, the `m_vec_mul_add` subroutine is repeatedly invoked during the execution of `P1P1t_times_O` and updates an accumulator with secret-dependent values derived from the oil coefficients. This means that the nibble-sliced multiplication routine `m_vec_mul_add` executes $\texttt{m\_vec\_limbs} \times (v-1)$ times for a fixed oil coefficient in the oil matrix $\mathbf{O}$. Consequently, during a single execution of MAYO, the total number of calls to `m_vec_mul_add` is $v \times o \times \big(\texttt{m\_vec\_limbs} \times (v-1)\big)$. For the MAYO-SL1 parameter set, this evaluates to 240,240 invocations. Each call updates the accumulator with a secret-dependent value, and this update is written back to memory, which causes observable power leakage. This repeated and structured leakage directly enables a correlation-based attack on the accumulator. Therefore, a single execution of MAYOproduces 240,240 leakage samples of the accumulator for the same secret-dependent computation. This large number of samples is sufficient to capture the leakage behavior directly from the attack traces, and no separate profiling phase or reference device is required. To validate the practicality of this attack model, we next describe the power measurement setup used to acquire the traces.

Power measurement.

To measure power consumption, we use an experimental setup that consists of a CW308-STM32F3 target platform equipped with an ARM Cortex-M4 STM32F3SAM4S microcontroller operating at 24 MHz. Using this setup, we measure power consumption during the execution of the signing algorithm and isolate the time samples corresponding to the update and subsequent memory store of the accumulator. Although the accumulator is 64 bits in the software implementation, the underlying hardware registers are only 32 bits wide. In this context, our analysis focuses on the lower 32 bits of the accumulator. Although the upper 32 bits could also be incorporated, our experimental results show that full recovery is achieved across all experiments using only the traces corresponding to the lower 32 bits. Consequently, we restrict our analysis to the lower 32 bits throughout this work. The store operation following each accumulator update induces a power-expensive transition, making the leakage particularly prominent and easy to align across traces.

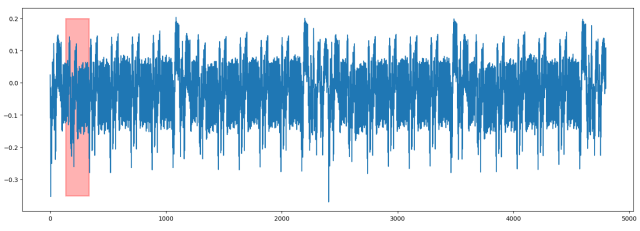

Figure 1: Temporal alignment of power traces using characteristic pre-computation spikes for multiplication segmentation.

The power consumption profile reveals distinct temporal markers that serve as reliable anchors for side-channel preprocessing, with the X-axis representing discrete sampling points and the Y-axis denoting the normalized signal amplitude. As illustrated in the concatenated trace in the Figure 1 , two prominent power spikes consistently prior to each multiplication operation. The red highlighted region identifies these multiplication operations. By utilizing these spikes as trigger points, we can perform precise trace segmentation and elastic alignment, effectively compensating for clock jitter or sampling offsets. This synchronization is a prerequisite for successful CPA. Since it ensures that the leakage from the subsequent `nibble-sliced` multiplications is perfectly aligned across the entire trace set.

Hypothetical intermediate computation. For each candidate value of the oil coefficient $\mathbf{O}(u,k) \in \mathbb{F}_{16}$, we compute the corresponding multiplication table `mul_table(a)` and evaluate the resulting accumulator update using the known public values `in[i]`. This yields hypothetical accumulator values for each key hypothesis.

```
Hypothetical_Leakage_Model(in, acc_init):
    // in        : public input vector (bitsliced)
    // acc_init  : known initial accumulator
    // returns   : hypothetical leakage for all key guesses

    for each key guess oil_coeff in F_16 do
        acc_hyp <- acc_init

        // run secret-dependent computation
        m_vec_mul_add(m_vec_limbs, in, oil_coeff, acc_hyp)

        // map intermediate state to leakage
        HypoLeak[oil_coeff] <- HW(acc_hyp)
    end for
return HypoLeak
```

Listing 2: Hypothetical leakage model for CPA

Leakage modeling. We map the hypothetical accumulator values to the predicted power consumption using the Hamming weight leakage model. Although individual nibble-wise multiplications exhibit limited distinguishability, the accumulated value combines multiple secret-dependent contributions, resulting in a sufficiently informative leakage signal.

Statistical analysis. In this phase, we compute the Pearson correlation coefficient (PCC) between the reference trace and the measured power traces for each key hypothesis. We observed that the subroutine `m_vec_mul_add` runs 240240 times for a single MAYO SL-1 execution, and it employs a total of 3120 different accumulators. Consequently, each oil-coefficient of $\mathbf{O}$ is involved in 385 separate `m_vec_mul_add` calls. It provides a rich set of sub-traces for statistical analysis.

As shown in Figure 2, the correct key hypothesis (highlighted in red) exhibits fast convergence. It quickly reaches a stable correlation within the first 25 sub-traces for the coefficient $O_{0,0}$. Conversely, incorrect hypotheses (indicated in gray) remain clustered around the zero-correlation baseline. This difference in correlation between the correct and incorrect keys helps recover the secret values correctly.

Figure 3 illustrates the maximum correlation across all hypothetical values for 385-sub traces. A prominent peak (marked in red) identifies the secret value as `0x05`, yielding the highest correlation. The substantial gap between the correct key and the other values helps to determine the potential secret oil coefficient. If the

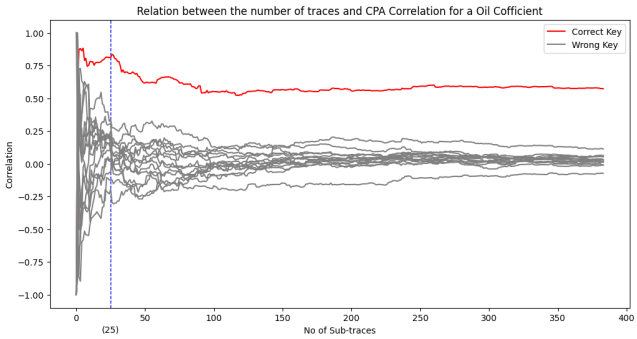

Figure 2: PCC over an increasing number of sub-traces for the coefficient $O_{0,0}$.

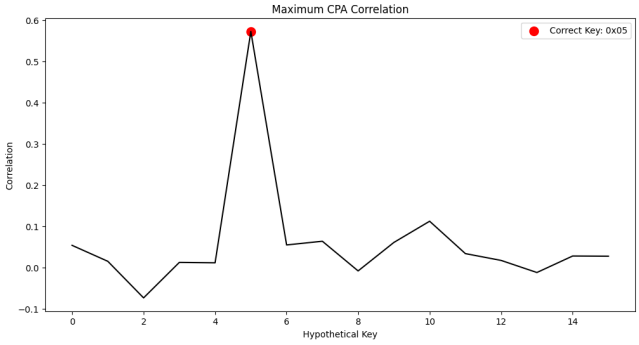

Figure 3: Maximum correlation for the coefficient $O_{0,0}$ across all key candidates.

peak is not clear, we use look-ahead mechanism. This technique give several likely guesses and only keeps the ones that remain mathematically consistent as the attack continues.

3.3 Recovering the Oil matrix

We discussed about the methodology to recover single coefficients of the oil-matrix $\mathbf{O}$ through CPA. Now, we discuss the way to recover all coefficients of the oil matrix using the mentioned CPA technique. Our entire recovery process has three steps: (i) *guess*, (ii) *verify*, and (iii) *update*.

(i) *The* guess *phase*. To recover the target coefficient $O_{r,c}$, we apply our CPA methodology to the interaction between the secret oil coefficients and the public $\mathbf{P}_i^{(1)}$ matrices. During experimentation, we observed that mispredictions frequently occur because the correlation of the actual secret key often fails to cross the required threshold. Even when the correct key produces the highest peak among all non zero candidates, its absolute correlation value remains below the tolerance limit. This makes it difficult to distinguish a valid key candidate from a zero value.

Consequently, when the correlation is negligible, the algorithm defaults to a zero value guess. An erroneous guess at one location causes the subsequent coefficients in the column to exhibit even lower correlation. This results in a chain reaction of consecutive

Algorithm 1 Recovering Oil Matrix $\mathbf{O}$

Require: Power traces $\{\mathcal{T}_i\}$, Public matrices $\mathbf{P}^{(1)}, \mathbf{P}^{(2)}$, Current accumulators acc
Ensure: Full Oil Matrix $\mathbf{O} \in \mathbb{F}_{16}^{v \times o}$

for $r = 0$ to $v - 1$ **do**
  for $c = 0$ to $o - 1$ **do**
    // Phase 1: Guess
    $\gamma \leftarrow \text{CPA}(\mathcal{T}, \text{acc}, \mathbf{P}^{(1)})$ ▷ Identify top candidate
    // Phase 2: Verify (Depth-1) ▷ Analyzing 0 vs γ case
    $acc_0 \leftarrow \text{acc} \oplus \text{Multiplication}(0, \mathbf{P}^{(1)})$
    $acc_\gamma \leftarrow \text{acc} \oplus \text{Multiplication}(\gamma, \mathbf{P}^{(1)})$
    $g_0 \leftarrow \text{CPA}(\mathcal{T}, acc_0)$
    $g_\gamma \leftarrow \text{CPA}(\mathcal{T}, acc_\gamma)$
    if $g_0 \neq 0$ and $g_\gamma = 0$ **then**
      $\mathbf{O}(r, c) \leftarrow 0$
    else if $g_0 = 0$ and $g_\gamma \neq 0$ **then**
      $\mathbf{O}(r, c) \leftarrow \gamma$
    else ▷ Ambiguity: Depth-2 Recursive Search
      for all Path $P \in \{(0,0), (0, g_0), (\gamma, 0), (\gamma, g_\gamma)\}$ **do**
        $acc_P \leftarrow \text{acc} \oplus \text{SimulatePath}(P)$
        $g_P \leftarrow \text{CPA}(\mathcal{T}, acc_P)$ ▷ Test on row $r + 2$
        if $g_P \neq 0$ **then**
          $\mathbf{O}(r, c) \leftarrow$ Root branch of P
          break
    // Phase 3: Update Accumulator
    $\gamma_{final} \leftarrow \mathbf{O}(r, c)$
    for $idx = 0$ to $v - 1, idx \neq c$ **do**
      $p \leftarrow (idx < c)?p_{idx,c} : p_{c,idx}$
      $\text{acc}_{idx,c} \leftarrow \text{acc}_{idx,c} \oplus (p \times \gamma_{final})$
return $\mathbf{O}$

zero guesses because the internal state of the simulation no longer matches the hardware.

However, since the matrix $\mathbf{O}$ is generated randomly, finding many zeros in a column consecutively is statistically unlikely. If the algorithm identifies too many consecutive zeros due to low correlation, we assume a misprediction has occurred. To fix this, we keep two candidates for the position: the zero value and the top ranked CPA guess. We then use a Verify Phase to test both candidates and determine which one allows the attack to continue successfully.

(ii) *The* verify *phase.* To resolve ambiguities between legitimate secret coefficients and thresholding errors, we implement a *look-ahead verification* algorithm with a search depth of two. This process determines the actual secret value for $O_{r,c}$ by evaluating its impact on upcoming recovery steps. The algorithm utilizes the initial guess of 0, the candidate guess γ obtained from the CPA methodology, the corresponding column of the public matrix $\mathbf{P}_i^{(1)}$, and the current state of the accumulator *acc*. The verification begins by initializing two distinct accumulators. We update the first accumulator, acc_0, assuming a secret value of `0x00`. Simultaneously, we update the second accumulator, acc_γ, using the candidate guess γ. Then we attempt to recover the next sequential coefficient, $O_{r+1,c}$, using both states.

Let $guess_0$ denotes the result obtained using acc_0, and $guess_\gamma$ denotes the result obtained using acc_γ. The secret value for $O_{r,c}$ is determined based on the success of these future recoveries. If $guess_0 \neq 0$ and $guess_\gamma = 0$, we conclude that the synchronization is only maintained by the zero-path; thus, the secret value for $O_{r,c}$ is 0. Also, if $guess_0 = 0$ and $guess_\gamma \neq 0$, the state synchronization holds only for the CPA candidate path; therefore, we conclude the secret value for $O_{r,c}$ is γ. This decision logic is summarized in Table 2.

Table 2: Look-Ahead Verification for Secret Determination

Guess	Accumulator	Guess at $O_{r+1,c}$	Determining Secret
0	acc_0	$guess_0$	If $guess_0 \neq 0$, $guess = 0$
γ	acc_γ	$guess_\gamma$	If $guess_\gamma \neq 0$, $guess = \gamma$

At this stage, the recovery of $O_{r+1,c}$ remains ambiguous because both $guess_0$ and $guess_\gamma$ appear equally likely (either both are zero or both are non-zero). To resolve this, the search is extended to the next coefficient, $O_{r+2,c}$. Since each branch ($guess_0$ and $guess_\gamma$) introduces two further possibilities, we evaluate a total of four potential paths to predict the value of $O_{r+2,c}$. For every potential path, we update the accumulators to a specific transition state and apply the CPA methodology to get a tentative secret key for $O_{r+2,c}$.

Table 3: Depth-2 Recursive Verification Matrix with State Transitions

Guess at $O_{r,c}$	Accumulator State (i)	Guess at $O_{r+1,c}$	Accumulator State ($i+1$)	Guess at $O_{r+2,c}$
0	acc_0	0	acc_{0_0}	$guess_{acc_{0\_0}}$
0	acc_0	$guess_0$	acc_{0_g0}	$guess_{acc_{0\_g0}}$
γ	acc_γ	0	acc_{γ_0}	$guess_{acc_{\gamma\_0}}$
γ	acc_γ	$guess_\gamma$	acc_{γ_γ}	$guess_{acc_{\gamma\_\gamma}}$

The correct secret value for the original target, $O_{r,c}$, is identified by determining which path at position $(r+2, c)$ results in a non-zero recovery. If a non-zero guess is detected within a specific branch, we conclude that the initial guess at row r which initiated that path is the true secret value.

The Table 3 explains the fourth possible paths. These four potential paths are $(0, acc_0) \rightarrow (0, acc_{0\_0}) \rightarrow guess_{acc_{0\_0}}$, $(0, acc_0) \rightarrow (guess_0, acc_{0\_g0}) \rightarrow guess_{acc_{0\_g0}}$, $(\gamma, acc_\gamma) \rightarrow (0, acc_{\gamma\_0}) \rightarrow guess_{acc_{\gamma\_0}}$, and $(\gamma, acc_\gamma) \rightarrow (guess_\gamma, acc_{\gamma\_\gamma}) \rightarrow guess_{acc_{\gamma\_\gamma}}$. To illustrate, consider the third path, where the candidate guess for $O_{r,c}$ is γ. We first update the current accumulator state with this value, resulting in the intermediate state acc_γ. To verify this branch at a second-order depth, we assume the next coefficient $O_{r+1,c}$ is 0 and update the accumulator accordingly to reach the state acc_{γ_0}. Using this specific state, we apply the CPA methodology to predict the hypothetical key value for $O_{r+2,c}$. If this prediction, $guess_{acc_{\gamma\_0}}$, yields a non-zero value, it confirms that the initial guess γ at position $O_{r,c}$ is the correct secret value.

(iii) *The* Update *phase.* At this moment, we able to identify the secret key. Once the oil coefficient $O_{r,c}$ is recovered, we update the accumulator. Then, we follow the same approach to recover $O_{r,c+1}$, and thus recover the entire $\mathbf{O}$.

Here, we provide a detailed breakdown of the update mechanism. Following the formulation in Equation 1, the matrix $\mathbf{L}_i$ is initialized with the coefficients of $\mathbf{P}_i^{(2)}$, where the (r, c) entry is denoted as $acc_{r,c}$. During the computation, the $\mathbf{O}$ matrix coefficient $O_{r,c}$ is multiplied by $(v-1)$ distinct elements of the symmetric matrix $(\mathbf{P}_i^{(1)} + \mathbf{P}_i^{(1)\top})$ and accumulated into $\mathbf{L}_i$. It means $O_{r,c}$ interacts with the set $\{p_{0,c}, \ldots, p_{c-1,c}, p_{c,c+1}, \ldots, p_{c,v-1}\}$, where each product updates corresponding accumulators $acc_{0,c}, \cdots, acc_{c-1,c}, \cdots acc_{c+1,c}, acc_{v-1,c}$.

$$\mathbf{L}_i \leftarrow \begin{pmatrix} acc_{0,0} & \cdots & acc_{0,o-1} \\ acc_{1,0} & \cdots & acc_{1,o-1} \\ \vdots & \ddots & \vdots \\ acc_{v-1,0} & \cdots & acc_{v-1,o-1} \end{pmatrix}$$

$$\mathbf{L}_i \leftarrow \mathbf{L}_i + \begin{pmatrix} 0 & p_{0,1} & \cdots & p_{0,v-1} \\ p_{0,1} & 0 & \cdots & p_{1,v-1} \\ p_{0,2} & p_{1,2} & \cdots & p_{2,v-1} \\ \vdots & \vdots & \ddots & \vdots \\ p_{0,v-1} & p_{1,v-1} & \cdots & 0 \end{pmatrix} \times \begin{pmatrix} O_{0,0} & \cdots & O_{0,o-1} \\ O_{1,0} & \cdots & O_{1,o-1} \\ \vdots & \ddots & \vdots \\ O_{v-1,0} & \cdots & O_{v-1,o-1} \end{pmatrix} \tag{3}$$

During the recovery of $O_{r,c}$, the power traces leak the transitions of these accumulators as they ingest the hypothesized product. Once the secret value is identified, the entire set of accumulators is synchronized by adding the product of the public coefficients and the recovered $O_{r,c}$. This update process is generalized by the following state transition:

$$\left(acc_{idx,c} \xleftarrow{\text{Update}} acc_{idx,c} + \sum_{idx>c} p_{c,idx} \times \gamma + \sum_{idx<c} p_{idx,c} \times \gamma \right)_{idx=0, idx\neq c}^{v-1} \tag{4}$$

After synchronizing the accumulator states, the algorithm proceeds to the next column position, $O_{r,c+1}$. This sequence is repeated for all coefficients in the Oil matrix $\mathbf{O}$.

Validation through an experiment. Figure 4 illustrates the correlation coefficients for three distinct key hypotheses: `0x02` (blue), `0x09` (green), and texttt0x0 (yellow). This figure demonstrates that non-zero keys produce prominent, easily identifiable correlation peaks, whereas the zero-key correlation remains flat with no clear pattern. Furthermore, the peaks for the non-zero keys are highly localized and sharp at their respective values. This precision ensures that different non-zero candidates are clearly distinguishable from one another and easily separated from the zero-value baseline, effectively eliminating ambiguity during the secret recovery process.

4 SCA on UOV, QR-UOV and SNOVA

In this section, we extend our single trace non-profiled SCA to other MQ-DSAs, like UOV [8], QR-UOV [13], and SNOVA [27]. The implementation for SNOVA and QR UOV utilizes a compositional structure. In contrast, UOV [8] uses a structure based on subspaces similar to MAYO. The underlying field for QR UOV is $\mathbb{F}_{251}$. We conduct each experiment within a simulated environment containing significant noise levels.

4.1 Single Trace Non-profiled SCA on UOV

For parameter selection purpose, UOV specification [8] uses two different set of parameters, one $\mathbb{F}_{16}$, and another is $\mathbb{F}_{256}$. The $\mathbb{F}_{16}$ is only used in `uov-Is` parameters, and $\mathbb{F}_{256}$ is used for all security

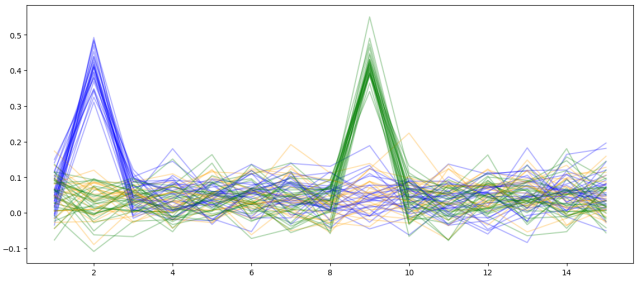

Figure 4: CPA correlation for keys `0x02` (blue), `0x09` (green), and `0x00` (yellow); sharp non-zero peaks contrast with the flat zero-key profile to resolve recovery ambiguity.

levels. The multiplication `gf16v_mul_u32` is used for $\mathbb{F}_{16}$ multiplication, and the subroutine `gf256v_mul_u32` is used for $\mathbb{F}_{256}$ field multiplication. We analyse the leakage point for both multiplications, and the cause behind leakages.

Leakage points for `gf256v_mul_u32`.[1] From a SC, the output of this subroutine is the primary leakage point, as it aggregates multiple secret-dependent operations within the multiplication loop. The key source of leakage is the update `r32 ^= a32 * ((b32 >> i) & 1)` where the accumulator `r32` undergoes a state transition only when the i-th bit of the secret multiplier b equals one. This conditional update introduces a clear data-dependent switching behavior.

At the hardware level, the Hamming distance of `r32` between successive iterations depends on the secret-dependent evolution of `a32`. Consequently, power or electromagnetic leakage observed when `r32` is updated exhibits a high signal-to-noise ratio, enabling an attacker to distinguish multiplier bits based on the presence or absence of observable switching activity. Hence, the output of the multiplication constitutes a natural and effective leakage point under standard side-channel leakage models. y the same reasoning, the output of the subroutine `gf16v_mul_u32` is the main leakage point $\mathbb{F}_{16}$ multiplication.

For `uov-Ip` parameters [8], $n = 112$ $m = 44$, $v = n - m = 68$, and $q = 256$. The subroutine `gf256v_mul_u32` is invoked a total of 603 times per oil coefficients, resulting in 1,804,176 invocations during a single UOV-`uov-Ip` signing operation. Consequently, a sufficient number of sub-traces is available, making a non-profiled single-trace attack on UOV feasible. Once a single oil coefficient is recovered, the sequential nature of the underlying matrix multiplication allows the attacker to progressively recover the entire oil matrix.

5 Conclusion

We showed a novel SCA on the implementations of MQ-DSAs. Even a basic attack setup is sufficient to challenge the security of these schemes. Notably, our attack can be carried out using only commodity hardware.

To protect cryptographic implementations against side-channel attacks, a widely used countermeasure is masking. An interesting future direction for this work is to explore the extent to which

[1] UOV GitHub: `gf16.h` file

masking MQ-DSA can protect against non-profiled/profiled single-trace side-channel attacks.

References

[1] Gorjan Alagic, Daniel Apon, David Cooper, Quynh Dang, Thinh Dang, John Kelsey, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, Dustin Moody, Rene Peralta, Ray Perlner, Angela Robinson, and Daniel Smith-Tone. 2022. Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process. Online. Accessed 26th January, 2024. https://nvlpubs.nist.gov/nistpubs/ir/2022/NIST.IR.8413-upd1.pdf

[2] Thomas Aulbach, Fabio Campos, and Juliane Krämer. 2025. SoK: On the Physical Security of UOV-Based Signature Schemes. In *Post-Quantum Cryptography*, Ruben Niederhagen and Markku-Juhani O. Saarinen (Eds.). Springer Nature Switzerland, Cham, 199–231.

[3] Thomas Aulbach, Fabio Campos, Juliane Krämer, Simona Samardjiska, and Marc Stöttinger. 2023. Separating Oil and Vinegar with a Single Trace Side-Channel Assisted Kipnis-Shamir Attack on UOV. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2023, 3 (2023), 221–245. doi:10.46586/TCHES.V2023.I3.221-245

[4] Ward Beullens. 2021. Improved Cryptanalysis of UOV and Rainbow. In *Advances in Cryptology – EUROCRYPT 2021*, Anne Canteaut and François-Xavier Standaert (Eds.). Springer International Publishing, Cham, 348–373.

[5] Ward Beullens, Fabio Campos, Sofia Celi, Basil Hess, and Matthias J. Kannwischer. 2023. *MAYO C implementation.* Available at https://github.com/PQCMayo/MAYO-C. Accessed June, 2023.

[6] Ward Beullens, Fabio Campos, Sofía Celi, Basil Hess, and Matthias J. Kannwischer. 2024. Nibbling MAYO: Optimized Implementations for AVX2 and Cortex-M4. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2024, 2 (Mar. 2024), 252–275. doi:10.46586/tches.v2024.i2.252-275

[7] Ward Beullens, Fabio Campos, Sofia Celi, Basil Hess, and Matthias J. Kannwischer. 2025. MAYO Specification Document Version 2.0. https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-2/spec-files/mayo-spec-round2-web.pdf

[8] Ward Beullens, Ming-Shing Chen, Jintai Ding, Boru Gong, Matthias J. Kannwischer, Jacques Patarin, Bo-Yuan Peng, Dieter Schmidt, Cheng-Jhih Shih, Chengdong Tao, and Bo-Yin Yang. 2025. UOV: Unbalanced Oil and Vinegar Algorithm Specifications and Supporting Documentation Version 2.0. https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-2/spec-files/uov-spec-round2-web.pdf

[9] Ward Beullens, Ming-Shing Chen, Shih-Hao Hung, Matthias J. Kannwischer, Bo-Yuan Peng, Cheng-Jhih Shih, and Bo-Yin Yang. 2023. Oil and Vinegar: Modern Parameters and Implementations. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2023, 3 (June 2023), 321–365. doi:10.46586/tches.v2023.i3.321-365 Artifact available at https://artifacts.iacr.org/tches/2023/a11.

[10] Jintai Ding and Dieter Schmidt. 2005. Rainbow, a New Multivariable Polynomial Signature Scheme. In *ACNS (Lecture Notes in Computer Science, Vol. 3531)*. 164–175.

[11] Leo Ducas, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehle. 2017. CRYSTALS – Dilithium: Digital Signatures from Module Lattices. Cryptology ePrint Archive, Paper 2017/633. https://eprint.iacr.org/2017/633 https://eprint.iacr.org/2017/633.

[12] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. 2020. Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU. https://falcon-sign.info/falcon.pdf.

[13] Hiroki Furue, Yasuhiko Ikematsu, Fumitaka Hoshino, Tsuyoshi Takagi, Haruhisa Kosuge, Kimihiro Yamakoshi, Rika Akiyama, Satoshi Nakamura, Shingo Orihara, and Koha Kinjo. 2025. QR-UOV Specification Document. https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/qruov-spec-round2-web.pdf

[14] Karine Gandolfi, Christophe Mourtel, and Francis Olivier. 2001. Electromagnetic Analysis: Concrete Results. In *Cryptographic Hardware and Embedded Systems — CHES 2001*, Çetin K. Koç, David Naccache, and Christof Paar (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 251–261.

[15] Sönke Jendral and Elena Dubrova. 2024. Single-trace side-channel attacks on MAYO exploiting leaky modular multiplication. Cryptology ePrint Archive, Paper 2024/1850. https://eprint.iacr.org/2024/1850

[16] David S Johnson and Michael R Garey. 1979. *Computers and Intractability: A Guide to the Theory of NP-completeness.* WH Freeman.

[17] Aviad Kipnis, Jacques Patarin, and Louis Goubin. 1999. Unbalanced Oil and Vinegar Signature Schemes. In *Advances in Cryptology — EUROCRYPT '99*, Jacques Stern (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 206–222.

[18] Aviad Kipnis and Adi Shamir. 1998. Cryptanalysis of the oil and vinegar signature scheme. In *Advances in Cryptology — CRYPTO '98*, Hugo Krawczyk (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 257–266.

[19] Paul Kocher, Joshua Jaffe, and Benjamin Jun. 1999. Differential Power Analysis. In *Advances in Cryptology — CRYPTO' 99*, Michael Wiener (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 388–397.

[20] Paul C. Kocher. 1996. Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems. In *Advances in Cryptology — CRYPTO '96*, Neal Koblitz (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 104–113.

[21] Suparna Kundu, Quinten Norga, Angshuman Karmakar, Uttam Kumar Ojha, Anindya Ganguly, and Ingrid Verbauwhede. 2024. mUOV: Masking the Unbalanced Oil and Vinegar Digital Signature Scheme at First- and Higher-Order. Cryptology ePrint Archive, Paper 2024/1875. https://eprint.iacr.org/2024/1875

[22] NIST. 2024. FIPS 204 Module-Lattice-Based Digital Signature Standard. Online. Accessed 10th November, 2024. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.204.pdf

[23] NIST. 2024. FIPS 205 Stateless Hash-Based Digital Signature Standard. Online. Accessed 10th November, 2024. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.205.pdf

[24] NIST. 2024. PQC digital signature second round announcement. Online. Accessed 10th November, 2024. https://csrc.nist.gov/News/2024/pqc-digital-signature-second-round-announcement

[25] Aesun Park, Kyung-Ah Shim, Namhun Koo, and Dong-Guk Han. 2018. Side-channel attacks on post-quantum signature schemes based on multivariate quadratic equations. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2018, 3 (Aug. 2018), 500–523. doi:10.13154/tches.v2018.i3.500-523

[26] David Pokorný, Petr Socha, and Martin Novotný. 2021. Side-channel attack on Rainbow post-quantum signature. In *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 565–568. doi:10.23919/DATE51398.2021.9474157

[27] Lih-Chung Wang, Chun-Yen Chou, Jintai Ding, Yen-Liang Kuan, Jan Adriaan Leegwater, Ming-Siou Li, Bo-Shu Tseng, Po-En Tseng, and Chia-Chun Wang. 2025. SNOVA Specification Document Version 2.0. https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-2/spec-files/snova-spec-round2-web.pdf